



VISION LLM · BUSINESS APPLICATIONS

แชทบอท กับ Vision LLM เพื่อการประยุกต์ใช้ธุรกิจ



ภาพรวมบทเรียน



ภาคที่ 1

ทฤษฎี Vision LLM

- ประเภทงานและภูมิทัศน์โมเดล Vision LLM
- กลไกการทำงานของ Cloud Inference
- การรันโมเดลแบบ Local / On-Premise
- กรอบตัดสินใจและข้อจำกัดเชิงทฤษฎี



ภาคที่ 2

ปฏิบัติ — Workshop

- องค์ประกอบและสถาปัตยกรรมของระบบ
- ทำไมต้องรู้เรื่องนี้และประโยชน์การใช้งาน
- หลักการวิศวกรรมซอฟต์แวร์ที่ใช้จริง
- Workshop สร้างระบบทีละขั้นและแบบฝึกหัด



วัตถุประสงค์การเรียนรู้

เมื่อจบบทเรียนนี้ ผู้เรียนควรทำสิ่งต่อไปนี้ได้

- 01 **อธิบายหลักการ VLM** — อธิบายหลักการทำงานเบื้องหลังของ Vision-Language Model ได้ในระดับแนวคิด
- 02 **เลือก Cloud หรือ Local** — เปรียบเทียบข้อดีข้อเสียของการ inference แบบ cloud กับ local เพื่อเลือกใช้ให้เหมาะกับองค์กร
- 03 **ออกแบบ Prompt สกัดข้อมูล** — ออกแบบ prompt สำหรับงานสกัดข้อมูลเชิงโครงสร้างจากภาพ โดยระบุกรณีกำกวมไว้ล่วงหน้า
- 04 **สร้างและ Deploy ได้จริง** — สร้างและ deploy web service ที่เชื่อมต่อกับ vision-LLM ผ่าน API ได้จริงด้วย FastAPI และ Docker



ประเภทงานที่ Vision LLM ทำได้

- 01 **Image Captioning** — การบรรยายภาพโดยรวม
- 02 **Visual Question Answering** — การตอบคำถามเกี่ยวกับเนื้อหาในภาพ
- 03 **OCR** — การอ่านตัวอักษรจากภาพ
- 04 **Document & Chart Understanding** — การทำความเข้าใจเอกสารและกราฟ
- 05 **Structured Extraction** — การสกัดข้อมูลเชิงโครงสร้าง



ภูมิทัศน์โมเดลเปิดน้ำหนัก (Open-Weight)

ดาวน์โหลดมารันเองได้ — ตัวเลขเปลี่ยนเร็ว ควรตรวจสอบซ้ำก่อนใช้สอนแต่ละครั้ง

โมเดล	ขนาดพารามิเตอร์	ผู้พัฒนา	จุดเด่น
Qwen2.5-VL-72B / 7B	72B / 7B	Alibaba	คะแนน OCR สูงสุดในกลุ่มเปิด
InternVL3-78B / 8B	78B / 8B	Shanghai AI Lab	สัญญาอนุญาต MIT เปิดกว้าง
Llama 4 Maverick / Scout	400B / 109B (17B active)	Meta	สถาปัตยกรรม Mixture-of-Experts
Phi-4-Multimodal	5.6B	Microsoft	ขนาดเล็ก รันบนฮาร์ดแวร์จำกัดได้
Pixtral 12B	12B	Mistral	สัญญาอนุญาต Apache 2.0
MiniCPM-V / Molmo	8B / 72B, 7B	— / Allen AI	เน้นความเปิดเผยของข้อมูลฝึกและโค้ด



Cloud Inference — เส้นทางของภาพจนถึง Vision Encoder

OpenRouter Docs ระบุว่า image_url รับได้ทั้ง URL และภาพเข้ารหัส Base64



← หลังถอดรหัสสมบูรณ์เท่านั้น ภาพจึงถูกตัดเป็น patch เข้าสู่ vision encoder



Cloud Inference — จุดแข็งและจุดอ่อน

- 01 จุดแข็ง** — ไม่ต้องลงทุนฮาร์ดแวร์ เข้าถึงโมเดลคุณภาพสูงสุดของวงการได้ทันที และได้รับอัปเดตรุ่นใหม่โดยอัตโนมัติ
- 02 จุดอ่อน** — ข้อมูลภาพต้องถูกส่งออกจากเครือข่ายภายในไปยังผู้ให้บริการภายนอกเสมอ — ประเด็นสำคัญสำหรับหน่วยงานที่ทำงานกับเอกสารอ่อนไหว เช่น เอกสารราชการหรือทางทหาร



Local / On-Premise Inference — เครื่องมือ

รันโมเดลเปิดน้ำหนักรบนฮาร์ดแวร์ขององค์กรเอง ข้อมูลไม่ต้องออกจากเครือข่ายภายใน

01 **Ollama** — รองรับโมเดลกลุ่ม vision ผ่านระบบ library ของตนเอง เช่น llama, llama3.2-vision, qwen2/3-vl

02 **LMDeploy** — เน้นรันโมเดล vision-language ร่วมกับเทคนิคบีบอัดแบบ INT4 บนฮาร์ดแวร์ระดับองค์กร



Local Inference — ความต้องการฮาร์ดแวร์ตามขนาดโมเดล

ขนาดโมเดล	ตัวอย่าง	ฮาร์ดแวร์ขั้นต่ำ
เล็ก (~5-6B)	Phi-4-Multimodal	GPU ขนาด 16GB
กลาง (~7-8B)	Qwen2.5-VL-7B	GPU ขนาด 24GB ตัวเดียว เช่น RTX 4090
ใหญ่ (~70B+)	Qwen2.5-VL-72B	GPU หลายตัวร่วมกัน หรือบีบอัดให้รันบน GPU เดี่ยวขนาด 80GB



Local Inference — จุดแข็งและจุดอ่อน

- 01 จุดแข็ง** — ข้อมูลไม่ออกจากองค์กร และเมื่อลงทุนฮาร์ดแวร์แล้วต้นทุนการใช้งานต่อครั้งแทบไม่มีเพิ่มอีก ต่างจาก cloud ที่คิดค่าใช้จ่ายตามปริมาณการใช้งาน
- 02 จุดอ่อน** — ต้องลงทุนฮาร์ดแวร์ล่วงหน้าเป็นจำนวนมาก ต้องดูแลระบบเอง และคุณภาพของโมเดลเปิดน้ำหนักรวมยังตามหลังโมเดลปิดชั้นนำอยู่บ้าง



กรอบการตัดสินใจ: Cloud หรือ Local

หน่วยงานที่ทำงานกับข้อมูลอ่อนไหวสูงมักเอนไปทาง local แม้ต้นทุนสูงกว่า

มิติ	Cloud	Local
ต้นทุนเริ่มต้น	ต่ำมาก ไม่ต้องลงทุนฮาร์ดแวร์	สูง ต้องซื้อ GPU
ต้นทุนต่อการใช้งาน	จ่ายตามปริมาณ token	แทบไม่มีเพิ่มหลังลงทุนแล้ว
ความเป็นส่วนตัวของข้อมูล	ข้อมูลออกจากองค์กรเสมอ	ข้อมูลอยู่ภายในองค์กรทั้งหมด
คุณภาพผลลัพธ์สูงสุด	สูงสุดของวงการ ณ ขณะนั้น	ตามหลังโมเดลปิดชั้นนำบ้าง
ความเร็วในการเริ่มใช้งาน	เริ่มได้ทันที	ต้องจัดเตรียมฮาร์ดแวร์และระบบก่อน
ภาระการดูแลรักษา	ผู้ให้บริการดูแลให้	องค์กรต้องดูแลเอง



ข้อจำกัดเชิงทฤษฎีของ Vision LLM

ธรรมชาติของกลไกการทำงาน ไม่ใช่ข้อบกพร่องของผู้ให้บริการรายใดรายหนึ่ง

- 01 **Hallucination ในบริบทภาพ** — โมเดลอาจให้คำตอบที่ฟังดูมั่นใจแต่ไม่ตรงกับสิ่งที่ปรากฏในภาพจริง
- 02 **ความแม่นยำของลายมือเขียน** — ต่ำกว่าตัวอักษรพิมพ์อย่างมีนัยสำคัญ — พบจริงเมื่อทดสอบกับใบเสร็จที่เขียนด้วยลายมือ
- 03 **ต้นทุน token ตามความละเอียดภาพ** — ภาพความละเอียดสูงถูกตัดเป็นจำนวน patch ที่มากขึ้น ทำให้ต้นทุนประมวลผลเพิ่มโดยอัตโนมัติ



PART 2 · ปฏิบัติ

ภาคปฏิบัติ

เวิร์กช็อปสร้างระบบจริง

นำหลักการจากภาคทฤษฎีมาประยุกต์สร้างระบบแชทบอทอ่านใบเสร็จที่ deploy ใช้งานได้จริง

โปรเจกต์: ระบบแชทบอทอ่านใบเสร็จรับเงินอัตโนมัติ · FastAPI · OpenRouter · Docker · Render



Workshop: องค์ประกอบของระบบ

สี่ส่วนที่ทำงานร่วมกัน



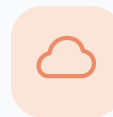
หน้าเว็บห้องแชท

รับข้อความและภาพ



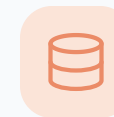
Backend FastAPI

ตัดสินใจโหมดประมวลผล



OpenRouter / Vision-LLM

อ่านภาพ สกัดข้อมูล



CSV + ภาพต้นฉบับ

บันทึกผล ตรวจสอบย้อนหลังได้

← ผลลัพธ์ JSON ส่งกลับเข้าห้องแชททันที



ทำไมต้องรู้เรื่องนี้

01 ปัญหาที่พบจริง — องค์กรจำนวนมากยังบันทึกรายจ่ายด้วยการพิมพ์ข้อมูลจากใบเสร็จด้วยมือ ใช้เวลานานและมีโอกาสผิดพลาดสูง

02 โอกาสที่เปิดขึ้นแล้ว — vision-LLM เข้าถึงได้ง่ายและราคาไม่สูงผ่าน API รวมอย่าง OpenRouter ทำให้ระบบอัตโนมัติเป็นไปได้จริงในต้นทุนที่องค์กรขนาดเล็กเข้าถึงได้



ประโยชน์การใช้งานและการต่อยอด

- 01 **ลดเวลาและความผิดพลาด** — ลดเวลาบันทึกข้อมูลและลดความผิดพลาดจากการพิมพ์ด้วยมือโดยตรง
- 02 **อัปเดตสู่ฐานข้อมูลจริง** — เปลี่ยนจากไฟล์ CSV ไปเป็นฐานข้อมูลจริงเมื่อปริมาณข้อมูลเพิ่มขึ้น
- 03 **เพิ่มการยืนยันตัวตน** — เพิ่มระบบยืนยันตัวตนผู้ใช้งานก่อนเปิดให้เข้าถึงจากภายนอก
- 04 **ต่อยอดเอกสารประเภทอื่น** — นำรูปแบบเดียวกันไปประยุกต์กับเอกสารที่มีโครงสร้างชัดเจน เช่น ใบลาหรือใบสั่งซื้อ ไม่จำกัดแค่ใบเสร็จ



หลักการวิศวกรรม — Prompt Engineering

ต้องระบุเงื่อนไขคำถามไว้ล่วงหน้า เพราะโมเดลไม่มีบริบทขององค์กรมาก่อน

01 **ชื่อผู้ออกใบเสร็จ vs ผู้จ่ายเงิน** — ใบเสร็จจำนวนมากมีทั้งชื่อองค์กรที่หัวกระดาษและชื่อผู้จ่ายเงินอยู่ในเอกสารเดียวกัน ต้องระบุให้ชัดว่าต้องการชื่อฝั่งไหน

02 **ยอดรวมทั้งสิ้น vs ยอดรับเงินมา** — ใบเสร็จที่มีการทอนเงินมักมีตัวเลขทั้งสองปรากฏอยู่ในเอกสารเดียวกัน ต้องระบุให้หยาบยอดรวมทั้งสิ้นเสมอ



หลักการวิศวกรรม — REST API ด้วย FastAPI

แยก endpoint ตามหน้าที่ ทดสอบและแก้ไขแต่ละส่วนได้อย่างอิสระจากกัน

01 **POST /api/chat** — รับข้อความและภาพเข้าห้องแชท ตัดสินใจว่าจะประมวลผลแบบถามตอบทั่วไปหรืออ่านใบเสร็จ แล้วเรียก vision-LLM

02 **GET /api/expenses** — ดึงรายการที่บันทึกไว้ใน CSV มาแสดงบน dashboard โดยเรียงรายการใหม่สุดไว้บนสุด



ข้อควรระวังและปัญหาที่พบจริง

ปัญหาจริงระหว่างการสร้างและทดสอบระบบ ไม่ใช่ปัญหาสมมติ

- 01 ปี พ.ศ. แปลงเป็น ค.ศ. ผิดพลาด** — ใบเสร็จไทยเขียนปีเป็นพุทธศักราช แก่ด้วยเกณฑ์ทั้งใน prompt และโค้ด: ปีมากกว่า 2400 ให้ลบ 543
- 02 โมเดลตอบไม่เป็น JSON บริสุทธิ์** — มีข้อความอธิบายหรือ markdown code fence ปนมา ต้องมีกลไกแยก JSON และเรียกซ้ำพร้อมย้าคำสั่งหากล้มเหลว
- 03 Filesystem ชั่วคราวบน Free Tier** — ข้อมูลที่บันทึกไว้จะหายไปทุกครั้งที่ไม่มีการใช้งานเกินเวลาที่กำหนดหรือมีการ deploy ใหม่



Workshop: เครื่องมือ

- 01 **FastAPI** — สร้าง web API ของระบบ
- 02 **OpenRouter** — เรียกใช้งาน vision-LLM ผ่าน API เดียวที่เชื่อมไปยังหลายผู้ให้บริการ
- 03 **Docker** — ห่อระบบให้พร้อม deploy ในสภาพแวดล้อมเดียวกันทุกที่
- 04 **Render** — โฮสต์ระบบให้เข้าถึงได้จากภายนอก



Workshop: โครงสร้างโปรเจกต์

แยกไฟล์ตามหน้าที่ ทำให้แก้ไขส่วนใดส่วนหนึ่งได้โดยไม่กระทบส่วนอื่น

ไฟล์ / ไฟลเดอร์	หน้าที่
main.py	แกนหลักของ backend
prompts/	เก็บ system prompt สองไฟล์ แยกโหมดถามตอบและโหมดอ่านใบเสร็จ
static/	เก็บหน้าเว็บห้องแชท
Dockerfile	สร้าง container สำหรับ deploy
requirements.txt	ระบุ dependency ที่ต้องติดตั้ง



Workshop: ขั้นตอนการสร้างระบบ

- 1 วางโครงไฟล์และตั้งค่า OpenRouter**
เชื่อมต่อ .env ชื่อโมเดลและ API key
- 2 เขียน system prompt สองไฟล์**
แยกโหมดถามตอบทั่วไปและโหมดอ่านใบเสร็จ
- 3 รับภาพและส่งเข้า vision-LLM**
แปลงภาพเป็น Base64 แล้วเรียก API
- 4 Normalize ข้อมูลที่ได้กลับมา**
แปลงวันที่ พ.ศ./ค.ศ. และแปลงจำนวนเงิน
- 5 บันทึก CSV และสร้างหน้าเว็บ**
เก็บภาพต้นฉบับพร้อมชื่อไฟล์ตรงกับ saved_at
- 6 ทดสอบ ห่อด้วย Docker และ Deploy**
ทดสอบกับใบเสร็จจริงหลายรูปแบบก่อนขึ้น Render



Workshop: การใช้งาน

01

อัปโหลดใบเสร็จจริง — สังเกตผลลัพธ์ JSON ที่ระบบสกัดออกมา

02

ถามคำถามเกี่ยวกับรายจ่าย — ดูว่าระบบดึงข้อมูลจาก CSV มาตอบได้ถูกต้องหรือไม่

03

ทดลองแก้ไข prompt — สังเกตความเปลี่ยนแปลงของผลลัพธ์เมื่อปรับคำสั่ง



สรุปเนื้อหา

- 01 Vision LLM = LLM + การแปลงภาพ** — ทำงานบนหลักการเดียวกับ LLM ทัวไป เพียงเพิ่มขั้นตอนแปลงภาพให้เป็น token ก่อนเข้าสู่กระบวนการเดิม
- 02 Cloud หรือ Local ขึ้นกับข้อมูลและงบ** — ความอ่อนไหวของข้อมูลและงบประมาณขององค์กรเป็นตัวตัดสินใจหลัก
- 03 ปฏิบัติเจอรายละเอียดที่ทฤษฎีไม่ครอบคลุม** — เช่น การจัดการปีพุทธศักราช ความน่าเชื่อถือของรูปแบบผลลัพธ์ และข้อจำกัดของแพลตฟอร์ม deploy ที่เลือกใช้



แบบฝึกหัดท้ายบท

- 01 ปรับ Prompt ให้เข้มงวดขึ้น** — ปรับ prompt ในไฟล์โหมดอ่านใบเสร็จ แล้วเปรียบเทียบผลลัพธ์กับใบเสร็จที่เคยอ่านผิดก่อนหน้านี้
- 02 ประเมินฮาร์ดแวร์สำหรับโมเดลเปิด** — เลือกโมเดลเปิดน้ำหนักหนึ่งตัวจากตารางภูมิทัศน์โมเดล แล้วประเมินว่าฮาร์ดแวร์ที่มีอยู่รันได้หรือไม่ พร้อมให้เหตุผล
- 03 ตัดสินใจ Cloud หรือ Local** — หากองค์กรของผู้เรียนต้องเลือกสำหรับงานประเภทนี้ ควรเลือกแบบใด พร้อมอธิบายเหตุผลโดยอ้างอิงกรอบการตัดสินใจ



ขอบคุณ

คำถาม & ข้อเสนอแนะ

ทดลองระบบจริงได้จากซอร์สโค้ดและตัวอย่างที่สาธิตในบทเรียนนี้

แชทบอท กับ Vision LLM เพื่อการประยุกต์เชิงธุรกิจ